Document Number: <QMS Ref. No.>	***Copy Number:*** <copy number>

TATA CONSULTANCY SERVICES

Trivandrum

Virtual ILP – Struts2 – Part 1

Content Manual

Version 1.0

May 2014

TATA CONSULTANCY SERVICES

A Division of **Tata Sons Limited**

DOCUMENT RELEASE NOTICE

Notice No.
Client
Project
Document details

Name	Version No.	Author	Description
Virtual ILP – Struts2 - Part1	1	Vidya R (199872)	Initial version of Struts 2

Revision details:

Action taken (add/del/	Preceding page No.	New page No.	Revision description

Change Register serial numbers covered:
The documents or revised pages are subject to document control.
Please keep them up-to-date using the release notices from the distributor of the document.
These are confidential documents. Unauthorized access or copying is prohibited.

Approved by : _____ Date: _____

Authorized by: _____ Date: _____

DOCUMENT REVISION LIST

Client
Project
Document Name
Release Notice Reference (for release)

Rev. No	Revision date	Revision description	Page No.	Prev page No.	Action taken	Addenda /New page	Release Notice reference

CONTENTS

COURSE 1 - STRUTS 2 – PART I 1

1.1 Objective 1
1.2 Course Content 1
1.2.1 Course Objectives 1
1.2.2 Introduction to Framework 1
1.2.2.1 Framework Architecture / Design Patterns 2
1.2.2.2 Model View Controller Architecture (MVC) 3
1.2.3 Advantages of Frameworks 4
1.2.3.1 Popular Frameworks 6
1.2.4 Introduction to Struts 6
1.2.4.1 History of Struts 6
1.2.4.2 Struts Latest Releases 7
1.2.4.3 Advantages of Struts framework 7
1.2.4.4 Prerequisites 8
1.2.5 Introduction to Struts 2 8
1.2.5.1 Features of Struts 2 8
1.2.5.2 Comparison of Struts 1.x and Struts 2.x 9
1.2.6 Struts2 Architecture 10
1.2.6.1 Struts2 Architecture Components in Detail 11
1.2.6.1.1 Model 12
1.2.6.1.1.1 Action 12
1.2.6.1.1.2 Steps to create Action in Struts2 12
1.2.6.1.2 Controller 14
1.2.6.1.2.1 Filters 14
1.2.6.1.2.2 Dispatcher Filters 17
1.2.6.1.2.3 Interceptors 17
1.2.6.1.3 View 18

COURSE 1 - STRUTS 2 – PART I

1.1 Objective

Objective of this document is to provide an overview of Struts framework and the various components that it supports.

1.2 Course Content

1.2.1 Course Objectives

At this point, you would have learnt how to develop a J2EE application using Servlets, JSP and JDBC.

In this document, would provide a detailed explanation of the following topics.

- What is a framework
- Advantages of using a framework in J2EE application development
- Introduction to Struts2 framework

1.2.2 Introduction to Framework

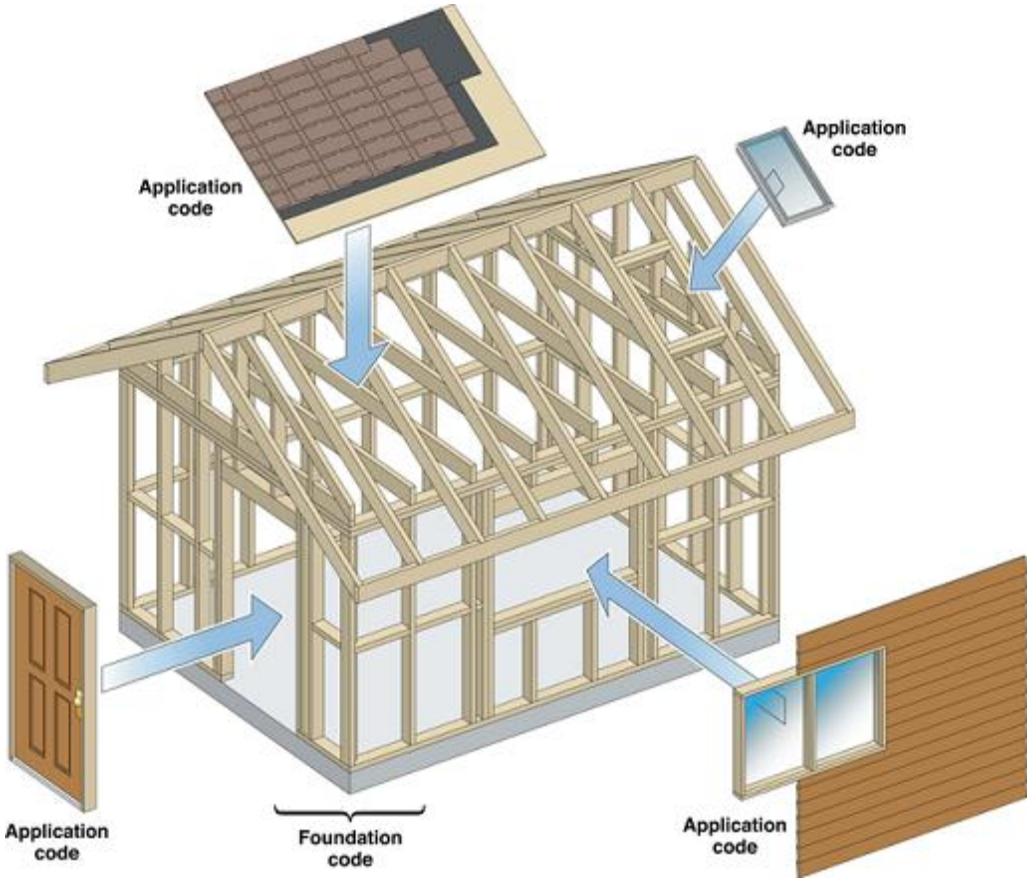
The word meaning of “framework” says “An essential supporting structure of a building, vehicle or object”.
In general, a framework is a real or conceptual structure intended to serve as a support or guide for the building of something that makes the structure into something useful.

Here we are going to discuss about Software frameworks which also serve as a support for developing a software application. Framework can be defined as an abstraction tool that makes it easier to develop applications and reuse the code.

A framework offers you many ways in which you can alter and extend its generic behaviour. It simply requires you to accept that all applications behave in the same fundamental ways because they are all based on the same structure.

In a broad metaphorical sense, a software framework is like the framework of a house, and your application code is like the doors, windows, siding, and other elements that make the house unique (Refer Pic:1).

Pic 1Understanding on Frameworks



Advantages of using frameworks

- Since all frameworks are developed based on good design principles and standards, by using the frameworks we can guarantee on the quality of the application that we develop.
- Increases the reliability and reduces the programming time since by using frameworks, we are re-using some code which has been already built and tested**.**

1.2.2.1 Framework Architecture / Design Patterns

"*A pattern is a proven solution to a problem in a context".*

An architecture pattern expresses the fundamental structural organization or schema for software systems. It defines a set of subsystems, specifies their responsibilities and includes the rules and guidelines for organizing the relationships between them.

Popular Architectural Patterns:

- **Layers**
- **Front Controller**
- **Model View Controller**
- **Multi tier Architecture**
- **Service Oriented Architecture**

and many more..

http://en.wikipedia.org/wiki/Architectural_pattern

1.2.2.2 Model View Controller Architecture (MVC)

Before MVC was introduced, the below issues were observed during application development.

- Frequent changes on the user interfaces without changing the business logic and vice versa
- As technologies change rapidly, you may want to support the same "look and feel" standards
- Requirement comes to change the core code from the interfaces

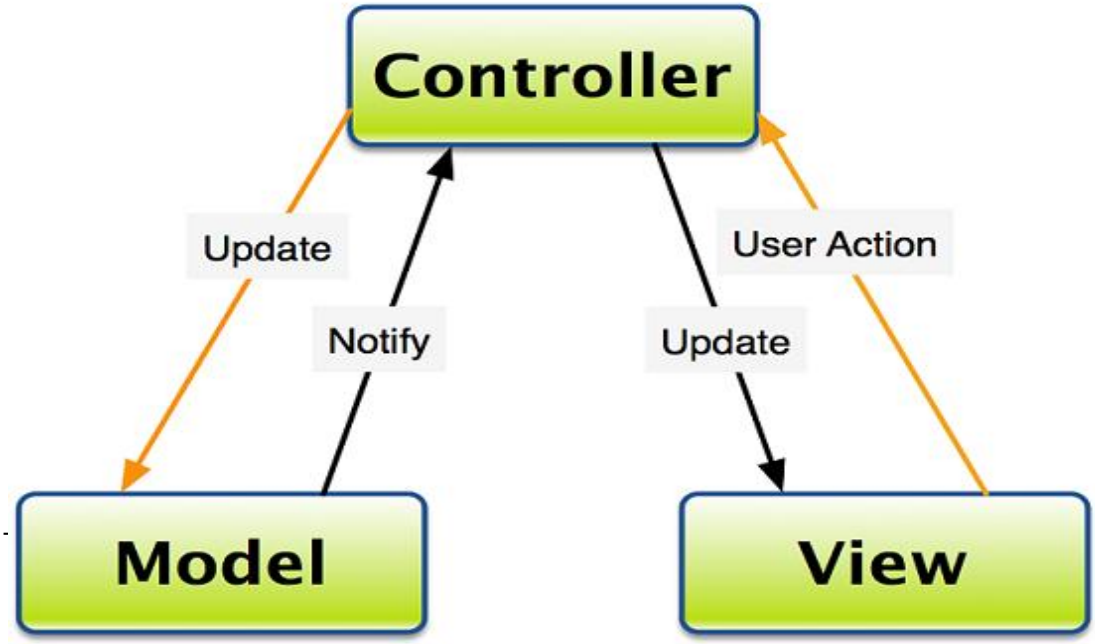
The above listed problems were addressed to a great extent by introducing MVC patterns.

MVC pattern use the software engineering principle of "separation of concerns" to divide the application into three areas:

Model encapsulates the core data and business logic/functionality

View encapsulates the presentation of the data. There can be different presentation for the same data.

for



Pic 2: MVC Architecture

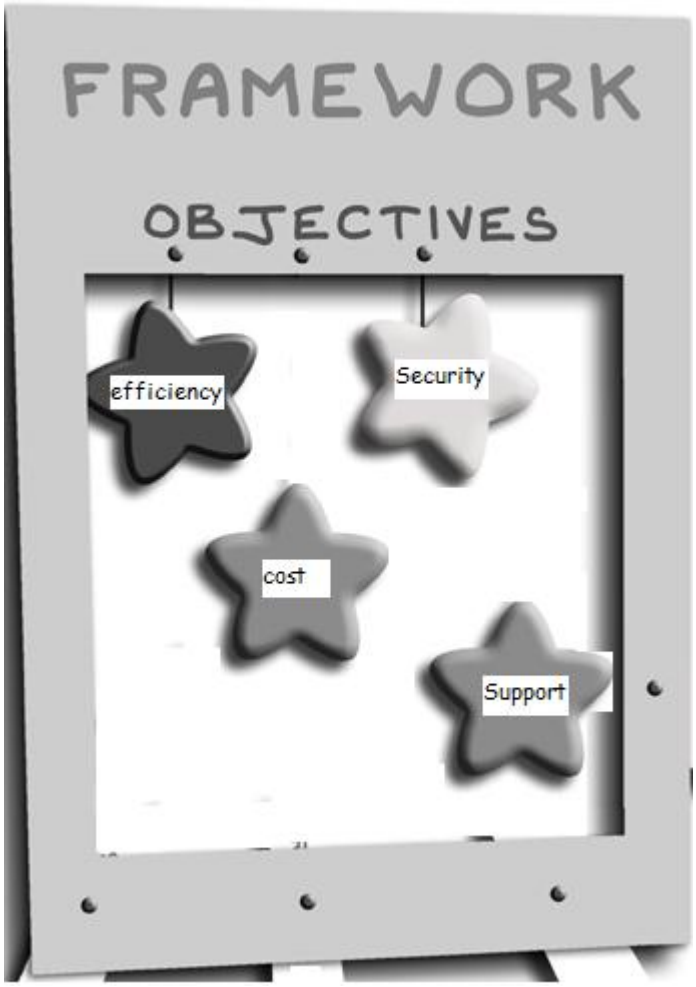
As you see in the above pic, View and Model should not interact directly and they should be loosely coupled as possible. This preserves re usability.

If any changes are required in the View, as per this design we can bring the change without Model being notified. Same is the case with any changes at the Model. Hence considering the changes requested in the J2EE applications over a period of time, its preferred to go with MVC architecture.

1.2.3 Advantages of Frameworks

Sometimes you should ask yourself, why you need to do the same thing every day when there's a new bridge to shorten the path. Think of a framework as a bridge that will help you code faster and easier.

Below given is a list of advantages that you will find in your project development when using a framework.



Pic 3: Advantages of a Framework

Efficiency
Tasks that usually would take you hours and hundreds of lines of code to write, can now be done in minutes with pre-built functions. Development becomes a lot easier, so if it's easier it's faster, and consequently efficient.

Security
A widely used framework has big security implementations. The big advantage is the community behind it, where users become long-term testers. If you find a vulnerability or a security hole, you can go to the framework's website and let the team know so they can fix it.

Cost
Most popular frameworks are free, and since it also helps the developer to code faster, the cost for the final client will be smaller.

Support
As any other distributed tool, a framework usually comes with documentation, a support team, or big community forums where you can obtain quick answers.

1.2.3.1 Popular Frameworks

Struts

Spring

JSF

and many more

Complete list of frameworks

1.2.4 Introduction to Struts

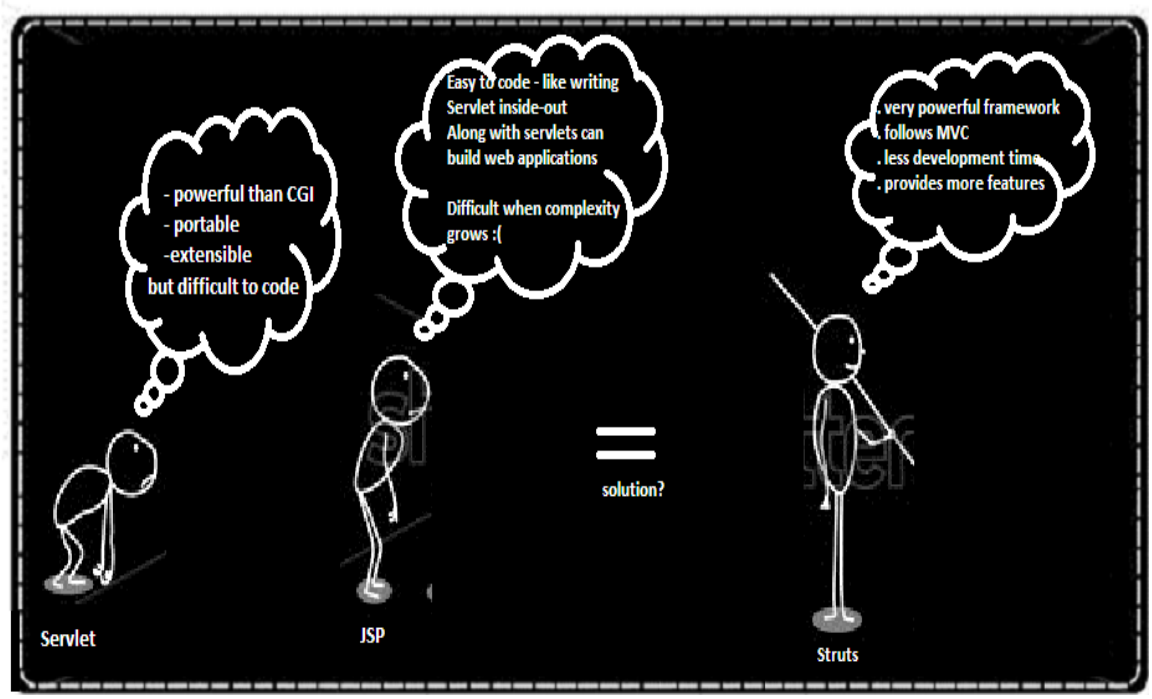
1.2.4.1 History of Struts

When Java Servlets were first invented, many programmers realized that they were good. They were faster and more powerful than~~t~~ standard CGI, portable, and infinitely extensible. But writing a lot of HTML code in a servlet to design complicated User Interfaces was found as a very tedious task. The answer to that was Java Server Pages which turned Servlet writing inside-out.

It is when applications started to have repetition of code as a result of increased business complexity, the need for a framework became more relevant.

The Apache Struts Project was launched in May 2000 by Craig R. McClanahan to provide a standard MVC framework to the Java community. In July 2001, version 1.0 was

released.



Pic 4: Evolution of Struts

1.2.4.2 Struts Latest Releases

Major Releases	Release Date
Struts 2.3.16.1	2. March 2014
Struts 2.2.3.1	7. September 2011
Struts 2.1.8	30. September 2009
Struts 2.0.6	22. February 2007
Struts 1.3.5	17. June 2006
Struts 1.2.4	19. September 2004
Struts 1.1	30. June 2003
Struts 1.0.2	11. February 2002

http://struts.apache.org/downloads.html

1.2.4.3 Advantages of Struts framework

- Struts 2.0 framwork has a feature called name spaces which makes bookmarking and managing URLs easy
- Struts framework provides a big set of Custom tag libraries to implement UI logic whithout writing java code inside scripletes in JSP.
- Provides powerful validation framework with which we can manage both client and server side validations.
- Multilingual support can be implemented easily through struts Internationalization(i18n) feature.
- It is easier to implement asynchronous service calls as the framework provides buil-int support for Ajax technology.

Apart from this, Apache has come up with a very good documentation on Struts compared to any other frameworks which will help the developer learn to use the framework.

http://struts.apache.org/release/2.0.x/docs/home.html

1.2.4.4 Prerequisites

- ☑ Knowledge on MVC
- ☑ Knowledge on Servlets & JSP

1.2.5 Introduction to Struts 2

Struts2 is popular and mature web application framework based on the MVC design

pattern. It is not just the next version of Struts 1, but it is a complete rewrite of the Struts architecture.

1.2.5.1 Features of Struts 2

Here are some of the great features that may force you to consider Struts2:

- **POJO Forms and POJO Actions**

  POJO stands for Plain Old Java Object. In the earlier versions of Struts (1.x), we used to write special “ActionForms” to receive form input and special “Action” classes to handle the user actions. With Struts2, you can use any POJO to receive form input and to process user actions.

- **Tag Support**

  Struts2 has improved the form tags and have come up with a set of new tags allow the developers to write less code

- **Ajax Support**

  Struts2 has come up with Ajax tags which will help developers to build Ajax features in their applications.

- **Easy Integration with other frameworks**

  Integration with other frameworks like Spring, Tiles and SiteMesh is now easier with a variety of integration available with Struts2.

- **Template Support**

  Support for generating views using different themes and templates available with Struts2.

- **Debug Support**

  Struts2 offers integrated debugging with the help of built in debugging tools.

- **Support for View Technologies**

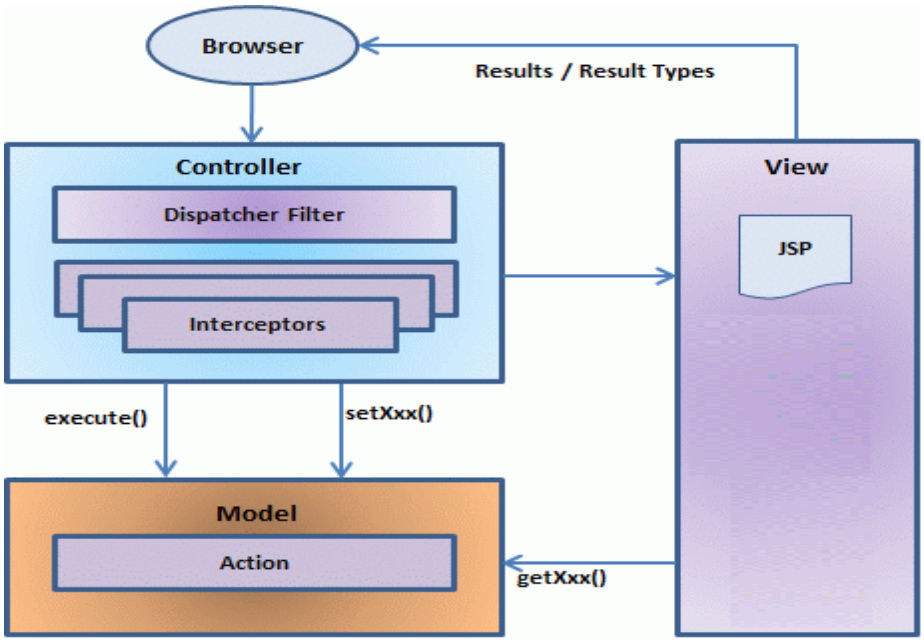
  Struts2 has a great support for multiple view technologies like JSP, XSLT, Velocity and freemarker.

1.2.5.2 Comparison of Struts 1.x and Struts 2.x

Feature	Struts2	Struts 1.x
Action/Action Form classes	Any POJO class can receive	Separate Action and Action

	form input and handle user actions	Form classes should be created
Thread Model	Struts 2 Action objects are instantiated for each request, so there are no thread-safety issues.	Struts 1 Actions are singletons and must be thread-safe since there will only be one instance of a class to handle all requests for that Action.
Servlet Dependency	Struts 2 Actions are not coupled to a container.	Struts 1 Actions have dependencies on the servlet API since the HttpServletRequest and HttpServletResponse is passed to the execute method when an Action is invoked.
Testability	Struts 2 Actions can be tested by instantiating the Action, setting properties, and invoking methods. Dependency Injection support also makes testing simpler.	A major hurdle to testing Struts 1 Actions is that the `execute` method exposes the Servlet API.
Expression Language	Struts 2 can use JSTL, but the framework also supports a more powerful and flexible expression language called "Object Graph Notation Language" (OGNL).	Struts 1 integrates with JSTL, so it uses the JSTL EL.
Type Conversion	Struts 2 uses OGNL for type conversion. The framework includes converters for basic and common object types and primitives.	Struts 1 ActionForm properties are usually all Strings. Struts 1 uses Commons-Beanutils for type conversion
Validation	Struts 2 supports manual validation via the `validate` method and the XWork Validation framework.	Struts 1 supports manual validation via a `validate` method on the ActionForm, or through an extension to the Commons Validator.
Control of Action Execution	Struts 2 supports creating different lifecycles on a per Action basis via Interceptor Stacks.	Struts 1 supports separate Request Processors (lifecycles) for each module, but all the Actions in the module must share the same life cycle.

1.2.6 Struts2 Architecture



As per the above diagram, the request life cycle of Strut2 can be explained as given below.

User sends a request to the server for requesting for some resource (Eg: Clicking a URL or submitting a form)

The FilterDispatcher looks at the request and then determines the appropriate Action.

Configured interceptors functionalities apply such as validation, file upload etc.
In the coming sections you will find more details about interceptors.

For the time being just understand that interceptors are a set of code that you want to execute before and after an action is invoked.

- Selected action is executed to perform the requested operation.
- Again, configured interceptors are applied to do any post-processing if required.
- Finally the result is prepared by the view and returns the result to the user.

1.2.6.1 Struts2 Architecture Components in Detail

As mentioned earlier, Struts2 is built based on MVC Architecture. So in order to understand the Struts2 architecture in detail, we need to know about the different components that come under MVC in Struts2.

1.2.6.1.1 Model

Model encapsulates the Core data and Business Logic. In Struts2 architecture model starts from Action Classes.

Do you know what an Action is?
Type in a URL in the address bar / click on a hyper link/ submit a form you are triggering an ac tion

1.2.6.1.1.1 **Action**

The Action layer in Struts2 has the below roles to perform

- Handles a basic "unit of work" for the application
- Serve as a Data Carrier
- Decision maker on what result to be invoked

Each request from the view is considered as an Action

Action transfers all the data from the request through the view (Will *see more about view later*).

Once the request processing is completely over the same Action layer decides on what result to be given to the View.

1.2.6.1.1.2 **Steps to create Action in Struts2**

- Every action is configured in struts.xml where the action class and Result is mapped
- Create POJO for every action class mapping in struts.xml

struts.xml is the main configuration file in Struts2 where all actions are configured.
Will see more about that in detail later

How can you create Action in Struts2?

Any POJO class can serve as Action class provided there must be one no-argument method that returns either String or Result object.

By default the no-argument method name is **execute**, but you can give any name to the method provided that method name is specified in struts.xml.

Please find below a sample Action class.

a simple POJO

```
public class CustomerAction
{
  private String customerName;
  public void setCustomerName(String customerName)
  {
        this.customerName = customerName;
  }
  public String getCustomerName()
  {
        return this.customerName;
  }
  public String execute
  {
        //communicate to other layers in model
        return "success";
  }
}
```

This is the method where action processing happens

1.2.6.1.2 Controller

As you know controller delegates the request from View to the appropriate Model and after processing the request the result is given back to the right view. In Struts2, the controller job is taken care by Dispatcher Filter and the request flow and response flow is intercepted by a layer known as Interceptors.

1.2.6.1.2.1 Filters

A filter is a layer that performs filtering tasks either on the request flow or to the response flow or both. We use filters commonly for the below purposes

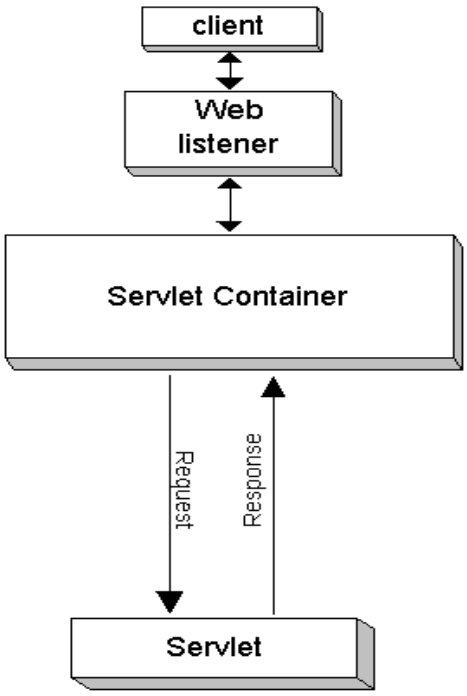
- Apply Authentication/Authorization check for request flows
- Apply Logging/Auditing feature for every request/response flow
- To encrypt request/response data flow

In web applications we make use of Servlet Filters to implement the filter concept.

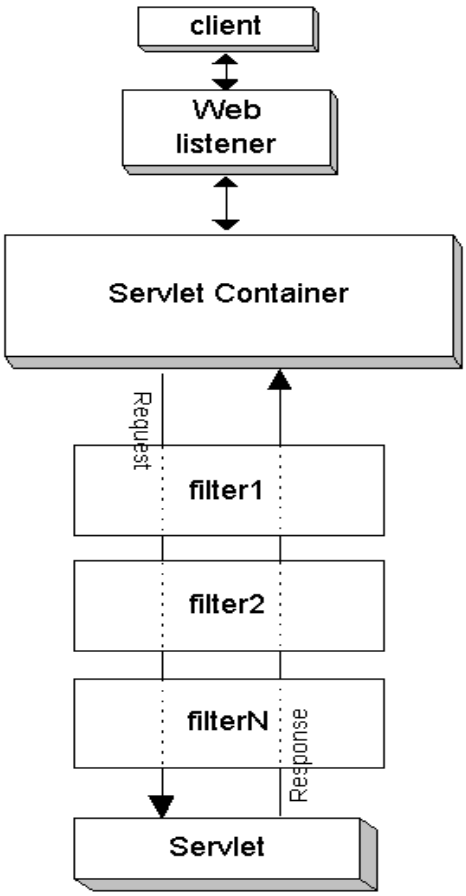
What is Servlet Filter and how to use it?

The picture that you see below shows the clear difference between request flow where no filters applied and request flow with filters applied.

In short if you have not configured filters, Servlet Container will directly pass the request to the Servlet for processing, whereas if filters are configured, Container invokes the filters one by one before it goes to the Servlet.



Flow without Filters configured



Flow when filters are configured

Similar to the Servlet interface there is an interface called Filter (javax.servlet.Filter) which has 3 methods as given below

- init()
- destroy()
- doFilter()

By comparing it with Servlet life cycle methods, you can very well understand that

init() – called by the container to indicate the filter that is being called

destroy() – to destroy the filter

doFilter() – to write the filtering logic

How to write a Filter and configure it?

How do you create a Servlet and configure it?
Like you create a Servlet, you have to implement Filter interface and implement the methods. Add entries in web.xml for the filter created

1. Create a class that implements Filter Interface and Implement the filter methods

```
package com.dev.filter.example;
import javax.servlet.*;
public class HelloWorldFilter implements Filter {
 private FilterConfig filterConfig;
 public void doFilter(final ServletRequest request, final ServletResponse response,
                FilterChain chain)
   throws java.io.IOException, javax.servlet.ServletException  {

  System.out.println("Entering  Filter");
  request.setAttribute("hello","Hello  World!");
  chain.doFilter(request,response);
  System.out.println("Exiting  HelloWorldFilter");
 }
 public void init(final FilterConfig filterConfig) {
  this.filterConfig = filterConfig;
 }
```

2. Add Filter entry in web.xml

```
<!-- Filter Example #1 -->
  <filter>
    <filter-name>helloWorld</filter-name>
    <filter-class>com.acme.filter.HelloWorldFilter</filter-class>
  </filter>

  <filter-mapping>
    <filter-name>helloWorld</filter-name>
    <url-pattern>/filter.jsp</url-pattern>
  </filter-mapping>

  <!-- end Filter Example #1 -->
```

1.2.6.1.2.2 Dispatcher Filters

So like any other filter you will have to configure DisptcherFilter in your web application.

DispatcherFilter code is available with Struts libraries so you just have to use it and configure it in web.xml.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://java.sun.com/xml/ns/javaee"
 <display-name>GreetStruts</display-name>
 <welcome-file-list>
  <welcome-file>index.html</welcome-file>
  <welcome-file>index.htm</welcome-file>
  <welcome-file>index.jsp</welcome-file>
  <welcome-file>default.html</welcome-file>
  <welcome-file>default.htm</welcome-file>
  <welcome-file>default.jsp</welcome-file>
 </welcome-file-list>
 <filter>
   <filter-name>struts2</filter-name>
   <filter-class>org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter</filter-class>
 </filter>
<filter-mapping>
   <filter-name>struts2</filter-name>
   <url-pattern>/*</url-pattern>
 </filter-mapping>
 </web-app>
```

1.2.6.1.2.3 Interceptors

Struts2 provides a set of interceptors which you can use for pre processing/post processing of request flow.

Popular interceptors are

Params – responsible for setting all request parameters to be set to the properties of the action

Validation – calls the validation framework and set the validation errors

FileUpload – Helps in uploading files

You will see more on interceptors in the coming sessions.

1.2.6.1.3 View

The main part of Struts2 View is driven by JSP.

Struts2 is providing a set of tag libraries which help us to implement the logic in JSP without writing Java code.

Three categories of tags are provided by Struts which are listed below

- **UI Tags** - every element rendered in the browser is implemented as a UI component in Struts.
- **Control Tags** are used to control the flow of the application.
- **Data Tags** are used to manipulate the data displayed on the page

1.2.7 Recap

Please find the recap of the main points that we discussed here

- What are a frameworks
- The advantages of using frameworks
- Struts2 Architecture
- Different components in Struts2